

Informe de Diseño de Base de Datos

Entrega 1 — Proyecto Wheels Alpes

Integrantes: David Felipe Mendoza Carrillo - 202421689 · Natalia Pulido Rodríguez 202421449
· Cesar Andrey Sanmiguel Mahecha — 202416049

Grupo: Grupo 9

Sección 1 — Análisis y Modelo Conceptual E/R

1.1 Contexto del negocio y entidades de negocio

Wheels Alpes es una plataforma de carpooling pensada exclusivamente para estudiantes universitarios. Conecta a estudiantes con vehículo propio (conductores) con estudiantes que necesitan transporte (pasajeros) en trayectos hacia o desde la universidad. Un conductor registra su información personal y al menos un vehículo; a partir de ahí puede publicar viajes indicando fecha y hora de salida, punto de origen, ruta, paradas disponibles, tarifa y estado. Los pasajeros pueden consultar los viajes disponibles filtrando por fecha y zona de recogida sin necesidad de autenticarse, pero para reservar un cupo deben autenticarse, elegir una de las paradas ofrecidas por el conductor y enviar una solicitud, que el conductor acepta o rechaza. El sistema también debe hacer cumplir reglas temporales de negocio — tanto el conductor como el pasajero pueden cancelar (viaje o solicitud, respectivamente) solo con un mínimo de dos horas de anticipación — y alimentar tableros de control con métricas históricas calculadas sobre los viajes completados.

Diccionario de entidades

Entidad	Tipo	Descripción
Estudiante	Superentidad	Comunidad universitaria registrada en la plataforma. Almacena los datos personales comunes a todo usuario: nombres, apellidos, documento de identidad, universidad, programa, correo institucional y número de celular.
Conductor	Subentidad (rol)	Especialización de Estudiante que ha inscrito al menos un vehículo y que puede publicar viajes y gestionar solicitudes de cupo.
Pasajero	Subentidad (rol)	Especialización de Estudiante que puede consultar viajes disponibles y enviar solicitudes de cupo.
Vehículo	Entidad básica	Medio de transporte inscrito por un conductor (placa, marca, modelo, año, color, cupos). Cada vehículo pertenece a un único conductor.
Viaje	Entidad básica	Itinerario publicado por un conductor usando uno de sus vehículos; agrupa fecha/hora de salida y llegada, ruta, tarifa y estado (programado, en curso, completado, cancelado).
Parada	Entidad débil	Punto intermedio de recogida/dejada propuesto por el conductor dentro de un viaje específico; no tiene sentido de existir fuera de un viaje.
SolicitudReserva	Entidad asociativa (tabla intermedia)	Resuelve la relación N:M entre Pasajero y Parada: representa la petición de un pasajero por un cupo en una parada concreta, con su estado de aceptación (Aceptado, Rechazado, Pendiente).

1.2 Reglas de negocio, cardinalidades y obligatoriedad

- Herencia con superposición: un Estudiante puede especializarse, de forma simultánea y opcional, como Conductor y/o Pasajero (un mismo estudiante puede ser ambos a la vez).
- Conductor (1,1) — posee — (1,N) Vehículo: todo conductor posee al menos un vehículo (es requisito para registrarse como conductor); un vehículo pertenece a exactamente un conductor.
- Vehículo (1,1) — se usa en — (0,N) Viaje: todo viaje usa exactamente un vehículo; un vehículo puede no haberse usado aún o haberse usado en muchos viajes.
- Viaje (1,1) — tiene — (1,N) Parada: toda parada pertenece a exactamente un viaje; todo viaje tiene al menos una parada.
- Pasajero (1,1) — envía — (0,N) SolicitudReserva: toda solicitud la envía exactamente un pasajero; un pasajero puede no haber enviado solicitudes aún o haber enviado muchas.
- Parada (1,1) — recibe — (0,N) SolicitudReserva: toda solicitud referencia exactamente una parada; una parada puede recibir cero o varias solicitudes de distintos pasajeros.

1.3 Diagrama Entidad-Relación (E/R)

El siguiente diagrama resume las entidades, atributos clave y relaciones descritas arriba, en notación Crow's Foot con el círculo "O" representando la herencia con superposición entre Estudiante y sus roles.

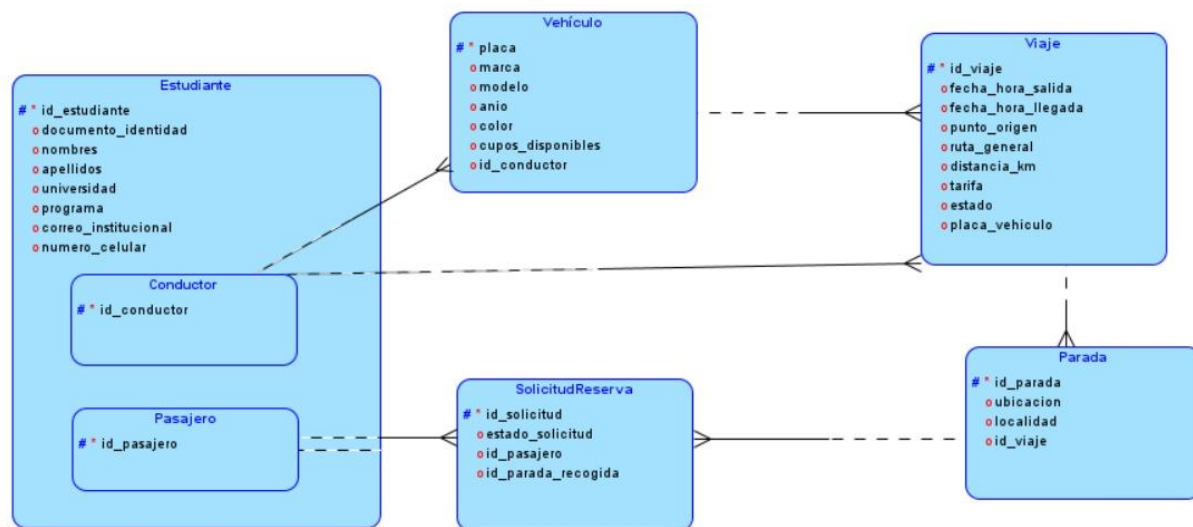


Figura 1. Diagrama conceptual E/R — Wheels Alpes.

1.4 Matriz de cobertura de requerimientos vs. modelo

Verificación de que el modelo conceptual soporta de forma natural cada operación exigida por el caso de estudio.

Req.	Descripción	Entidades involucradas	Cómo lo soporta el modelo
RF1	Registrar conductor	Estudiante, Conductor	El registro crea la tupla en Estudiante (datos personales) y la partición en Conductor (id_conductor = FK/PK hacia Estudiante).
RF2	Registrar vehículo	Vehículo	Nueva tupla en Vehículo con FK id_conductor hacia el conductor autenticado.
RF3	Publicar viaje	Viaje, Vehículo, Parada	Nueva tupla en Viaje referenciando el vehículo (placa) y una o más tuplas iniciales en Parada.
RF4	Gestionar solicitudes	SolicitudReserva, Parada, Viaje, Vehículo	Se actualiza el estado de la solicitud en la reserva y se agrega el viaje y la parada.

Req.	Descripción	Entidades involucradas	Cómo lo soporta el modelo
RF5	Actualizar estado del viaje	Viaje	Se actualiza el estado del viaje con necesidad de verificar que sea dos horas antes de la fecha programada de salida.
RF6	Consultar historial de viajes	Viaje, Parada, SolicitudReserva, Pasajero	No es propio de esta etapa pero se logra mediante el relacionamiento de las solicitudes con sus paradas con el id de viaje asignado a la parada.
RF7	Creación cuenta pasajero	Estudiante, Pasajero	El registro crea la tupla en Estudiante (datos personales) y la partición en Pasajero (id_pasajero = FK/PK hacia Estudiante).
RF8	Gestión cuenta pasajero	Estudiante	No es propio de esta etapa pero se logra accediendo a los datos del estudiante en su entidad.
RF9	Consultar viajes disponibles	Viaje, Parada	No es propio de esta etapa pero se logra accediendo a los viajes disponibles dependiendo su ruta y sus paradas asignadas.
RF10	Solicitar cupo en un viaje	SolicitudReserva	Nueva tupla con FK id_pasajero e id_parada, estado_solicitud = Pendiente.
RF11	Cancelar solicitud	SolicitudReserva, Parada, Viaje	No es propio de esta etapa pero se logra realizando la validación de la solicitud con su hora de inicio en el programa-
RFC1	Demanda por localidad y período	SolicitudReserva, Parada	Se logra accediendo a las columnas de viaje y parada relativo a la ubicación del usuario y a la fecha de salida del viaje.
RFC2	Ingresos por localidad y período	SolicitudReserva, Parada, Viaje	No es propio de esta etapa, pero se puede lograr mediante el calculo de viajes por su tarifa y su fecha de inicio.
RFC3	Viajes por conductor en el tiempo	Viaje, Vehículo, Conductor	Se puede lograr mediante el acceso el conteo de cada ID_viaje de conductor.
RFA1	Tablero de control — conductores	Viaje, Vehículo, SolicitudReserva, Parada	Se puede revisar mediante el acceso a las tuplas de los viajes, de las solicitudes en completado, de los periodos de tiempo buscados, etc.
RFA2	Tablero de control — pasajeros	SolicitudReserva, Parada, Viaje	Se puede acceder mediante las tuplas de acceso a las solicitudes realizadas por pasajero, a los viajes completados, a las paradas pedidas, etc.

Sección 2 — Diseño de Base de Datos Relacional y Justificación

2.1 Proceso de transformación E/R a relacional (Algoritmo de Chen)

Se aplicó el algoritmo de Peter Chen para convertir el modelo conceptual de la sección 1 en el esquema relacional de la sección 2.2, siguiendo las reglas de conversión vistas en el curso:

1. Entidades básicas

Estudiante, Vehículo, Viaje y Parada son entidades independientes y se convierten directamente en tablas, con sus atributos simples como columnas y su llave candidata más adecuada como PK. En Estudiante se identificaron tres llaves candidatas (`id_estudiante`, `documento_id`, se eligió `id_estudiante` como PK por ser una llave subrogada, estable y sin datos sensibles, dejando `documento_id` y `correo` como llaves alternas (nn, nd) disponibles para búsquedas. En Vehículo y Parada, las llaves naturales (`placa`) y subrogada (`id_parada`) respectivamente cumplen los criterios del curso para ser PK.

2. Herencia con superposición (Estudiante hacia Conductor / Pasajero)

Un estudiante puede ser conductor y pasajero al mismo tiempo (superposición). Se aplicó la regla de partición con PK/FK compartida: cada subentidad se modela como una tabla independiente cuyo único atributo es su identificador, que actúa simultáneamente como PK (identifica la fila del rol) y como FK hacia `Estudiante.id_estudiante` (referencia a los datos personales). Así, `id_conductor` y `id_pasajero` no son atributos nuevos: son el mismo `id_estudiante` del estudiante que asume ese rol. Esta estrategia evita duplicar nombre, documento, correo, etc. en cada rol, y permite que un mismo `id_estudiante` aparezca como fila en Conductor, en Pasajero, en ambas, o en ninguna.

3. Relaciones 1:N

La PK del extremo "1" migra como FK hacia la tabla del extremo "N":

- Conductor (1) — Vehículo (N): `id_conductor` migra a Vehículo como FK nn (todo vehículo tiene dueño obligatorio).
- Vehículo (1) — Viaje (N): `placa` migra a Viaje como FK nn (todo viaje usa un vehículo obligatorio).
- Viaje (1) — Parada (N): `id_viaje` migra a Parada como FK nn (dependencia de existencia: una parada no existe sin su viaje).

4. Relación N:M (Pasajero hacia Parada y viceversa)

La relación N:M entre Pasajero y Parada (un pasajero puede solicitar varias paradas; una parada puede recibir solicitudes de varios pasajeros) se resuelve creando la tabla intermedia llamada `SolicitudReserva`, que recibe las FK de ambas tablas (`id_pasajero`, `id_parada`) más su propia PK subrogada (`id_solicitud`) y el atributo propio de la relación (`estado_solicitud`).

2.2 Esquema relacional e inventario de restricciones

Estudiante (id_estudiante, documento_id, nombres, apellidos, universidad, programa, correo, numero_celular)

Atributo	Restricciones
id_estudiante	PK, NN, UA
documento_id	NN, ND
nombres	NN
apellidos	NN
universidad	—
programa	—
correo	NN, ND
numero_celular	NN

Conductor (id_conductor)

Atributo	Restricciones
id_conductor	PK, NN, ND, FK Estudiante.id_estudiante

Pasajero (id_pasajero)

Atributo	Restricciones
id_pasajero	PK, NN, ND, FK Estudiante.id_estudiante

Vehículo (placa, marca, modelo, anio, color, cupos, id_conductor)

Atributo	Restricciones
placa	PK, NN, UA
marca	NN
modelo	NN
anio	NN, [anio > 1900]
color	NN
cupos	NN, [cupos > 0]
id_conductor	FK Conductor.id_conductor, NN

Viaje (id_viaje, fecha_hora_salida, fecha_hora_llegada, punto_origen, ruta, distancia_km, tarifa, estado, placa)

Atributo	Restricciones
id_viaje	PK, NN, SA
fecha_hora_salida	NN
fecha_hora_llegada	NN, [fecha_hora_llegada > fecha_hora_salida]
punto_origen	NN
ruta	NN
distancia_km	NN, [distancia_km > 0]
tarifa	NN, [tarifa >= 0]
estado	NN, [('Programado','En curso','Completado','Cancelado')]
placa	FK Vehículo.placa, NN

Parada (id_parada, ubicacion, localidad, id_viaje)

Atributo	Restricciones
id_parada	PK, NN, SA
ubicacion	NN
localidad	NN
id_viaje	FK Viaje.id_viaje, NN

SolicitudReserva (id_solicitud, estado_solicitud, id_pasajero, id_parada)

Atributo	Restricciones
id_solicitud	PK, NN, SA
estado_solicitud	NN, [('Aceptado','Rechazado','Pendiente')]
id_pasajero	FK Pasajero.id_pasajero, NN
id_parada	FK Parada.id_parada, NN

2.3 Análisis de dependencias funcionales (DF) y llaves candidatas

Estudiante

Atributos: {id_estudiante (IE), documento_id (DOC), nombres (NOM), apellidos (APE), universidad (UNI), programa (PRO), correo (COR), numero_celular (CEL)}

Dependencias funcionales:

- IE → DOC, NOM, APE, UNI, PRO, COR, CEL
- DOC → IE, NOM, APE, UNI, PRO, COR, CEL

- $COR \rightarrow IE, DOC, NOM, APE, UNI, PRO, CEL$

Clausuras (X^+):

- $\{IE\}^+ = \{IE, DOC, NOM, APE, UNI, PRO, COR, CEL\} \rightarrow$ todos los atributos $\Rightarrow$ IE es superllave y, al ser un único atributo, es llave candidata.
- $\{DOC\}^+ =$ todos los atributos $\Rightarrow$ DOC es llave candidata.
- $\{COR\}^+ =$ todos los atributos $\Rightarrow$ COR es llave candidata.

Llaves candidatas: $\{id_estudiante\}$, $\{documento_id\}$, $\{correo\}$. **PK elegida:** $id_estudiante$ (llave subrogada).

Atributos primos: $id_estudiante$, $documento_id$, $correo$.

Atributos no primos: nombres, apellidos, universidad, programa, $numero_celular$.

Conductor y Pasajero

Atributos: $\{id_conductor\}$ y $\{id_pasajero\}$ respectivamente — una sola columna cada una.

Clausuras (X^+):

- $\{id_conductor\}^+ = \{id_conductor\}$ (y análogamente para $id_pasajero$) — por definición, el conjunto de todos los atributos de una relación es siempre superllave; al ser el único atributo, es también la llave candidata mínima.

Llave candidata de cada tabla: su único atributo ($id_conductor$ / $id_pasajero$).

Todos los atributos son primos (no hay atributos no primos: cada tabla tiene una sola columna).

Atributos no primos: no aplica (ninguno).

Vehículo

Atributos: $\{placa (PLA), marca (MAR), modelo (MOD), anio (ANI), color (COL), cupos (CUP), id_conductor (IDC)\}$

Dependencias funcionales no triviales:

- $PLA \rightarrow MAR, MOD, ANI, COL, CUP, IDC$
- IDC no deriva la placa por que un conductor puede tener uno o más vehículos.

Clausuras (X^+):

- $\{PLA\}^+ = \{PLA, MAR, MOD, ANI, COL, CUP, IDC\} \rightarrow$ todos los atributos $\Rightarrow$ PLA es la única llave candidata.

Llave candidata: $\{placa\}$. **PK:** $placa$ (llave natural).

Atributos primos: $placa$.

Atributos no primos: $marca, modelo, anio, color, cupos, id_conductor$.

Viaje

Atributos: {id_viaje (IV), fecha_hora_salida (FS), fecha_hora_llegada (FL), punto_origen (PO), ruta (RU), distancia_km (DK), tarifa (TA), estado (ES), placa (PLA)}

Dependencias funcionales:

- $IV \rightarrow FS, FL, PO, RU, DK, TA, ES, PLA$

Clausuras (X^+):

- $\{IV\}^+ = \{IV, FS, FL, PO, RU, DK, TA, ES, PLA\} \rightarrow$ todos los atributos

Llave candidata: {id_viaje}. PK: id_viaje (SA).

Atributos primos: id_viaje.

Atributos no primos: fecha_hora_salida, fecha_hora_llegada, punto_origen, ruta, distancia_km, tarifa, estado, placa.

Parada

Atributos: {id_parada (IP), ubicacion (UBI), localidad (LOC), id_viaje (IV)}

Dependencias funcionales:

- $IP \rightarrow UBI, LOC, IV$

Clausuras (X^+):

- $\{IP\}^+ = \{IP, UBI, LOC, IV\} \rightarrow$ todos los atributos $\Rightarrow$ IP es la única llave candidata.

Llave candidata: {id_parada}. PK: id_parada (SA).

Atributos primos: id_parada.

Atributos no primos: ubicacion, localidad, id_viaje.

SolicitudReserva

Atributos: {id_solicitud (IS), estado_solicitud (EST), id_pasajero (IPa), id_parada (IPd)}

Dependencias funcionales:

- $IS \rightarrow EST, IPa, IPd$

Clausuras (X^+):

- $\{IS\}^+ = \{IS, EST, IPa, IPd\}$

Llave candidata: {id_solicitud}. PK: id_solicitud (SA).

Atributos primos: id_solicitud.

Atributos no primos: estado_solicitud, id_pasajero, id_parada.

2.4 Verificación y demostración de formas normales (FNBC)

Evaluación de cada relación sobre la escalera 1FN → 2FN → 3FN → FNBC, a partir del análisis de dependencias funcionales de la sección 2.3.

Relación	1FN	2FN	3FN	FNBC	Conclusión / transformación
Estudiante	Cumple	Cumple: las tres llaves candidatas son de un solo atributo, por lo que no puede existir dependencia parcial.	Cumple: no se identifican dependencias transitivas entre los atributos no primos (nombres, apellidos, universidad, programa, numero_celular son mutuamente independientes según las reglas de negocio).	Cumple: los tres determinantes de las DF no triviales (id_estudiante, documento_id, correo) son superllaves.	FNBC sin transformación.
Conductor / Pasajero	Cumple.	Cumple (llave de un atributo).	Cumple (no hay atributos no primos).	Cumple vacuamente: no existen DF no triviales que puedan violarla.	FNBC sin transformación.
Vehículo	Cumple.	Cumple (llave simple: placa).	Cumple: no hay evidencia de dependencia transitiva entre los atributos no primos.	Cumple: el único X es placa, que es la CK.	FNBC con corrección de eliminación de dependencia transnitiva de Id_conductor.
Viaje	Cumple.	Cumple (llave simple: id_viaje).	NO cumplía en el esquema original (id_viaje que lleva a placa que lleva id_conductor, transitiva) — SÍ cumple en el esquema final, tras retirar id_conductor.	Cumple en el esquema final: único X = id_viaje = CK.	Se aplicó la transformación de para retirar Id_Conductor de Viaje. Resultado final: FNBC.
Parada	Cumple.	Cumple (llave simple: id_parada).	Cumple bajo el supuesto documentado en 2.3 (no se asume ubicacion → localidad).	Cumple: único determinante = id_parada = CK.	FNBC sin transformación
SolicitudReserva	Cumple.	Cumple (llave simple: id_solicitud).	Cumple: no hay dependencias transitivas entre estado_solicitud, id_pasajero e id_parada.	Cumple: único determinante = id_solicitud = CK.	FNBC sin transformación

Sección 3 — Escenarios de Prueba para Calidad e Integridad de Datos

Estudiante

<i>Prueba</i>	<i>Tupla de prueba</i>	<i>Resultado esperado</i>
De cada estudiante se debe almacenar su información personal: documento de identidad, nombres, apellidos, universidad, programa, correo institucional y número de celular	(1, 1032456789, "Juan", "Pérez", "Uniandes", "Ingeniería Industrial", "juan.perez@uniandes.edu.co", 3001234567)	La tupla debe pasar porque se cumplen todas las restricciones NN y ND de la tabla.
El id_estudiante no puede ser nulo ni se puede duplicar porque es la llave primaria de la tabla	**poner la tupla además de la anterior: (1, 1099887766, "Carlos", "Nieto", "Uniandes", "Ingeniería Civil", "carlos.nieto@uniandes.edu.co", 3109988776),	No debe pasar porque se viola la restricción ND de id_estudiante
El documento de identidad de cada estudiante debe ser único, por lo que no pueden existir dos cuentas asociadas al mismo documento	**poner tupla además de la primera de la tabla: (3, 1032456789, "Laura", "Ruiz", "Uniandes", "Ingeniería Ambiental", "laura.ruiz@uniandes.edu.co", 3201122334	No debe pasar porque se viola la restricción ND del documento de identidad

Conductor

<i>Prueba</i>	<i>Tupla de prueba</i>	<i>Resultado esperado</i>
Un estudiante puede registrarse en la plataforma como conductor si el estudiante ya está registrado en el sistema	1. ** se crea primero un estudiante con id_estudiante = 1: (id_conductor = 1) 2. (id_conductor = 8)	1. Pasa porque el id_conductor referencia a un estudiante ya creado, es decir, se cumple la FK hacia estudiante 2. No debe pasar, viola la integridad porque id_conductor debe ser una referencia (FK) hacia estudiante
Cada estudiante solo puede registrarse una vez como conductor, id_conductor no se puede repetir porque es la llave primaria	** crear nuevamente: (id_conductor = 1)	No pasa porque viola las restricciones PK y ND dado que el estudiante con id_estudiante = 1 ya fue registrado como conductor

Pasajero

Prueba	Tupla de prueba	Resultado esperado
Un estudiante puede registrarse en la plataforma como pasajero si el estudiante ya está registrado en el sistema	** id_estudiante = 2 existente: (id_pasajero = 2)	Pasa porque el id_pasajero referencia a un estudiante ya creado, es decir, se cumple la FK hacia estudiante
Un estudiante ya registrado como pasajero no puede volverse a registrar como pasajero	** crear nuevamente: (id_pasajero = 2)	No pasa porque viola las restricciones PK y ND dado que el estudiante con id_estudiante = 2 ya fue registrado como pasajero

Vehículo

Prueba	Tupla de prueba	Resultado esperado
Para cada vehículo se debe almacenar la información de su placa, marca, año, color, número de cupos disponibles y conductor	** id_conductor = 2 existe: ("ABC123", "Mazda", "3", 2020, "Gris", 3, 1)	Pasa porque cumple las restricciones NN de marca, modelo, año, color y cupos, además de que id_conductor exista en la tabla conductor
El registro de un vehículo debe realizar lo un estudiante ya registrado como conductor	** con id_conductor = 10 inexistente: ("XYZ789", "Renault", "Logan", 2019, "Blanco", 4, 10)	No pasa porque se viola la integridad del modelo al no existir una referencia válida a Conductor
La placa no se repite entre vehículos	** con placa "ABC123" existente: ("ABC123", "Chevrolet", "Spark", 2021, "Azul", 4, 1)	No pasa porque viola la restricción de la PK, la placa de los vehículos es ND
Se debe almacenar obligatoriamente el número de cupos	("DEF456", "Renault", "Logan", 2019, "Blanco", null, 1)	No pasa porque se viola la restricción NN de cupos

Viaje

Prueba	Tupla de prueba	Resultado esperado
Para cada viaje se debe almacenar la información de la fecha y hora estimada de salida, fecha y hora de llegada, punto de origen, la ruta general, distancia en km, la tarifa, el estado inicial y el vehículo que lo realizará	** con placa del vehículo ABC123 ya registrada: (1, "2026-09-15 07:00", "2026-09-15 07:40", "Calle 116 #15-20", "Cedritos-Uniandes", 12.5, 6000, "Programado", "ABC123")	Pasa porque se cumplen todas las restricciones NN de los atributos de la tabla, además de que se tiene como placa como FK existente hacia Vehículo
El estado del viaje debe ser correspondiente a programado, en curso, completado o cancelado	(1, "2026-09-15 07:00", "2026-09-15 07:40", "Calle 116 #15-20", "Cedritos-Uniandes", 12.5, 6000, "Iniciado", "ABC123")	No pasa porque "Iniciado" no corresponde al cominio permitido para el atributo estado
Un viaje está asociado a un vehículo ya registrado	(2, "2026-09-15 07:00", "2026-09-15 07:40", "Calle 116 #15-20", "Cedritos-Uniandes", 12.5, 6000, "Iniciado", "NPR722")	No pasa porque viola la restricción de la FK, la placa debe estar referenciada a un vehículo existente en la plataforma
El di_viaje no debe ser igual al de ningún otro viaje porque es su llave primaria	** Con id_viaje = 1 ya registrado: (1, "2026-09-15 07:00", "2026-09-15 07:40", "Calle 116 #15-20", "Cedritos-Uniandes", 12.5, 6000, "Programado", "NPR722")	No pasa porque viola la restricción de la PK, el id_viaje es ND

Parada

Prueba	Tupla de prueba	Resultado esperado
Para cada parada se debe almacenar la información de id_parada, ubicación, localidad y id_viaje	** con un viaje con id_viaje = 1 ya registrado: (1, "Portal Norte", "Suba", 1)	Pasa porque se cumple la restricción NN de los atributos y id_viaje está asociado a un viaje registrado
Cada parada debe estar asociada a un viaje ya publicado por un conductor	** id_viaje = 22 inexistente: (2, "Portal 80", "Engativá", 22)	No pasa porque viola la restricción de la FK, el id_viaje debe existir en la plataforma
El id_parada no puede repetirse entre paradas porque es la llave primaria	** id_parada = 1 ya registrado: (1, "Portal Dorado", "Fontibón", 2)	No pasa porque viola la restricción de la PK, el id_parada es ND

SolicitudReserva

<i>Prueba</i>	<i>Tupla de prueba</i>	<i>Resultado esperado</i>
Toda solicitudReserva debe ser creada por un pasajero, seleccionando un punto de partida entre las paradas disponibles del viaje	** id_pasajero = 2 y id_parada = 1 ya registrados: (1, "Pendiente", 2, 1)	Pasa porque cumple con las restricciones de NN de los atributos, el estado es uno de los valores dentro del dominio del atributo y se cumple la integridad de referencia de las FK id_pasajero y id_parada
El estado de una solicitud solo puede corresponder a aceptado, rechazado o pendiente	(1, "No aceptado", 2, 1)	No pasa porque "No aceptado" no se encuentra dentro del dominio permitido para el atributo estado_solicitud
Para crear una solicitudReserva el pasajero debe estar registrado	** con id_pasajero = 40 no registrado: (2, "Pendiente", 40, 1)	No pasa porque viola la restricción de la FK id_pasajero, el pasajero debe haber sido registrado en la plataforma

Conclusiones

- El modelo conceptual cubre de forma directa los 16 requerimientos del caso de estudio (RF1–RF11, RFC1–RFC3, RFA1–RFA2), según la matriz de cobertura de la sección 1.4.
- El esquema relacional original de la plantilla del equipo tenía una dependencia funcional transitiva en Viaje ($\text{id_viaje} \rightarrow \text{placa} \rightarrow \text{id_conductor}$) y restricciones ND mal aplicadas en cinco llaves foráneas del lado "muchos". Ambos problemas se identificaron mediante el análisis de dependencias funcionales y se corrigieron.
- Tras las correcciones, las siete relaciones del modelo (Estudiante, Conductor, Pasajero, Vehículo, Viaje, Parada, SolicitudReserva) quedan en Forma Normal de Boyce-Codd (FNBC), sin redundancias ni anomalías de inserción, actualización o borrado.
- Los escenarios de prueba de la sección 3 cubren unicidad de PK, integridad referencial, dominios (CHECK) y las reglas de negocio temporales propias del caso de estudio (cancelación con anticipación mínima de 2 horas).

Referencias bibliográficas

- García-Molina, H., Ullman, J. D., & Widom, J. Database Systems: The Complete Book.
- Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks.

- Materiales de clase ISIS1511: presentaciones de Modelo Conceptual, Modelo Relacional, Conversión Lógica, Dependencias Funcionales y Normalización — Universidad de los Andes.
- Enunciado oficial: Entrega 1 — Diseño, proyecto Wheels Alpes, ISIS1511 — Fundamentos de Bases de Datos.